Python 程序设计语言

计算机基础

- 计算机特点
  - 运算速度快
  - 计算精确度高
  - 具有记忆和逻辑判断能力
  - 有自动控制能力

计算机常用的数制

- 二进制，十进制，八进制，十六进制

二进制	十进制	八进制	十六进制	二进制	十进制	八进制	十六进制
0000	0	0	0	1000	8	10	8
0001	1	1	1	1001	9	11	9
0010	2	2	2	1010	10	12	A
0011	3	3	3	1011	11	13	B
0100	4	4	4	1100	12	14	C
0101	5	5	5	1101	13	15	D
0110	6	6	6	1110	14	16	E
0111	7	7	7	1111	15	17	F

文本编码---ASCII码

- 字符编码(Character Code)是用二进制编码来表示字母、数字以及专门符号
- 普遍采用是ASCII(American Standard Code for Information Interchange)码，例如：

ASCII值	字符	ASCII值	字符	ASCII值	字符
32	(space)	64	@	96	、
33	!	65	A	97	a
34	"	66	B	98	b

编码的要素

- 码字：　　a
- 编码：　　97
- 位和字节表示：　　01100001

Unicode编码

- Unicode码是计算机科学领域里的一项业界标准,包括字符集、编码方案等。
- Unicode是为了解决传统的字符编码方案的局限而产生的，它为每种语言中的每个字符设定了统一并且唯一的二进制编码，以满足跨语言、跨平台进行文本转换、处理的要求。
- '\u4e16\u754c\u60a8\u597d' 代表汉字

  世界你好，

  "\u"表示Unicode码。

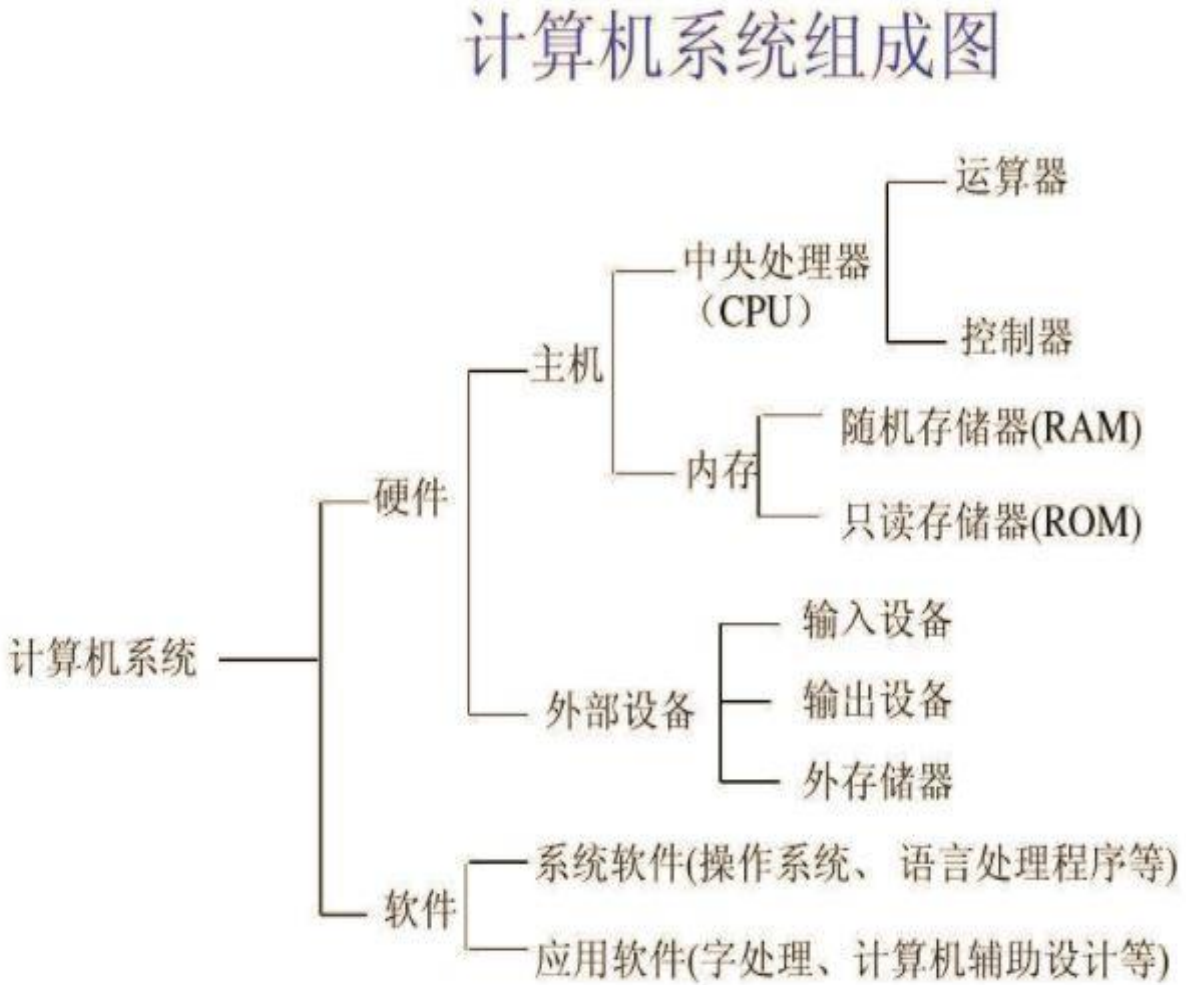
UTF-8编码

- Unicode码并不是存储器中的编码，使用时是把Unicode编码转换字节或位，UTF-8编码就起这个作用。

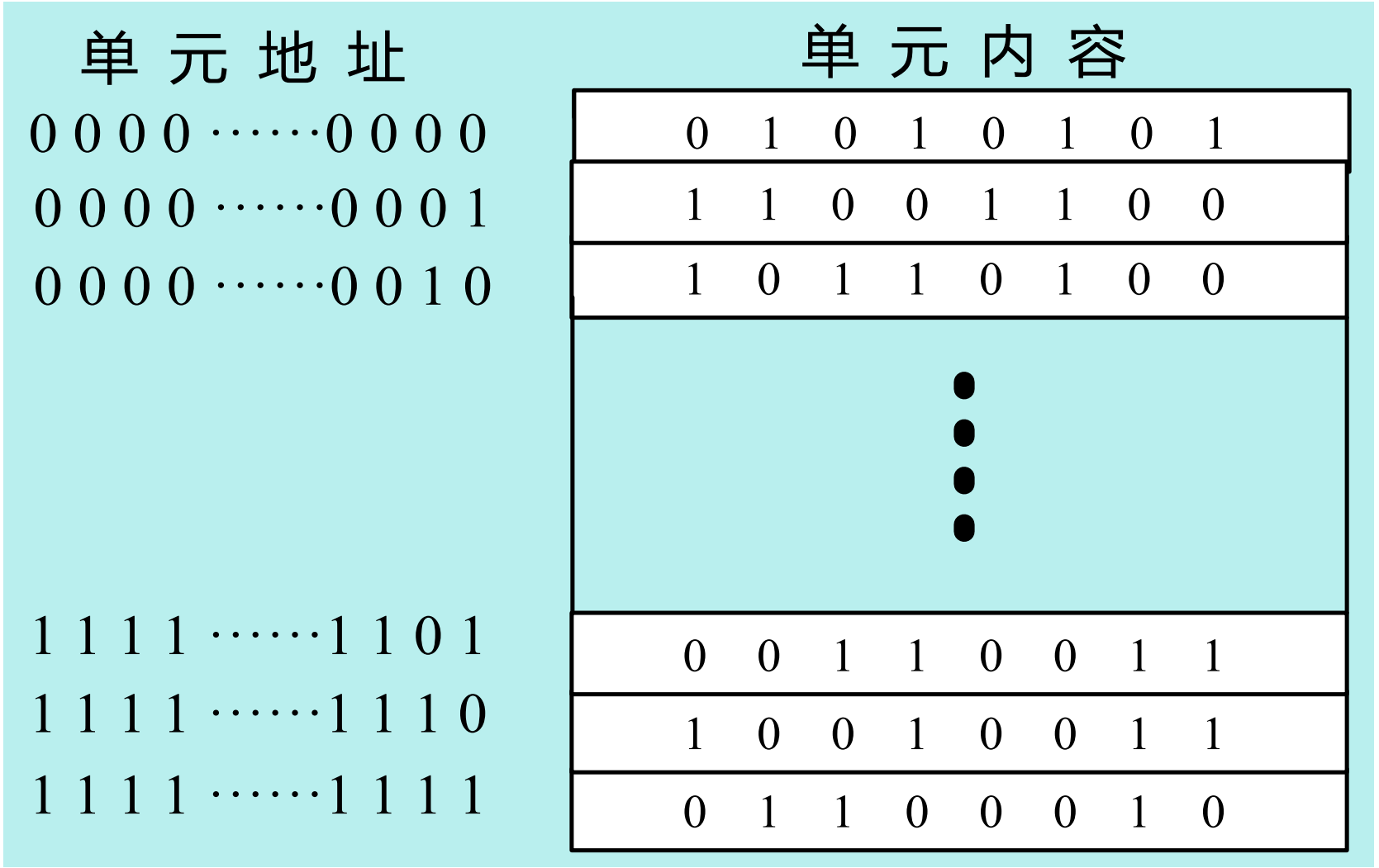
  每一个ASCII字符都有一个UTF-8编码，
  该UTF-8编码正好与8位的ASCII编码一样

  编写Python3程序，默认的是UTF-8编码·

计算机系统


计算机系统组成图
计算机系统
硬件
主机
中央处理器
（CPU）
运算器
控制器
内存
随机存储器(RAM)
只读存储器(ROM)
外部设备
输入设备
输出设备
外存储器
软件
系统软件(操作系统、语言处理程序等)
应用软件(字处理、计算机辅助设计等)

内存模型


单 元 地 址
单 元 内 容
0 0 0 0 ……0 0 0 0
0 0 0 0 ……0 0 0 1
0 0 0 0 ……0 0 1 0
1 1 1 1 ……1 1 0 1
1 1 1 1 ……1 1 1 0
1 1 1 1 ……1 1 1 1
0 1 0 1 0 1 0 1
1 1 0 0 1 1 0 0
1 0 1 1 0 1 0 0
0 0 1 1 0 0 1 1
1 0 0 1 0 0 1 1
0 1 1 0 0 0 1 0

存储单位

单位		实际字节数	近似表示方法
B(Byte)	字节	1	1
KB(K Byte)	千字节	2^{10}	10^3
MB(M Byte)	兆字节	2^{20}	10^6
GB(G Byte)	千兆字节(吉)	2^{30}	10^9
TB(T Byte)	兆兆字节(太拉)	2^{40}	10^{12}

操作系统

- 操作系统（Operating System，简称OS）是管理和控制计算机硬件与软件资源的计算机程序，是直接运行在“裸机”上的最基本的系统软件
- Unix/Linux
- Windows
- Mac OS

程序设计语言

- 机器语言
- 汇编语言
- 高级语言
- C/C++ 语言
- Java 语言
- Python 语言

为什么大数据首选是Python呢？

python™



离线界面型：Excel，Powerpoint，PS，Tableau……
离线代码型：R，SAS，Python，D3.js，Process……
在线型：Echarts，Tagxedo……

专用界面型：SPSS……
专用代码型：R，SAS……
泛用代码型：Python

数据库：SQL，Hive……
界面型：Excel，SPSS……
代码型：VBA，Python，R，SAS……

数据库：SQL，Hadoop，Hive……
爬虫：Python，Java……



第一部分 初识Python

Python语言的诞生和发展历史

Python语言的特点

运行环境及安装

Python语言的诞生和发展历史

- Python语言是一种解释型、面向对象、动态数据类型的高级程序设计语言
- Python语言是数据分析师的首选数据分析语言，也是智能硬件的首选语言

数据分析



动画电影效果

创建复杂的Web应用程序

.....

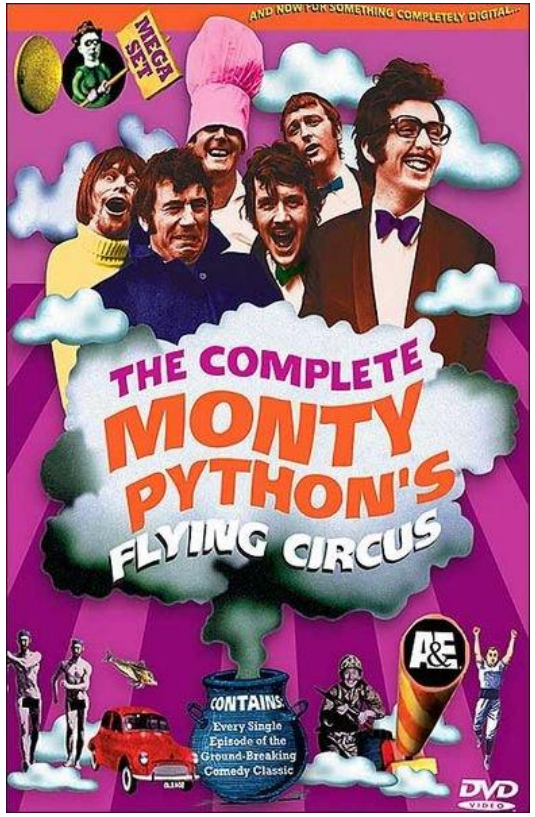
网站开发

游戏开发

智能硬件开发

Python语言的诞生

- Python与蟒蛇有关?
- Guido van Rossum 于1989年在荷兰国家数学和计算机科学研究所设计出来的
- Python的设计哲学是“优雅”、“明确”、“简单”
- Python是自由软件之一，免费、开源。
- Python已经被移植到许多平台上。这些平台包括Unix/Linux、Windows、Mac OS。



BBC “Monty Python‘s Flying Circus”（蒙提·派森的飞行马戏团）

Guido van Rossum（人称“龟叔”）



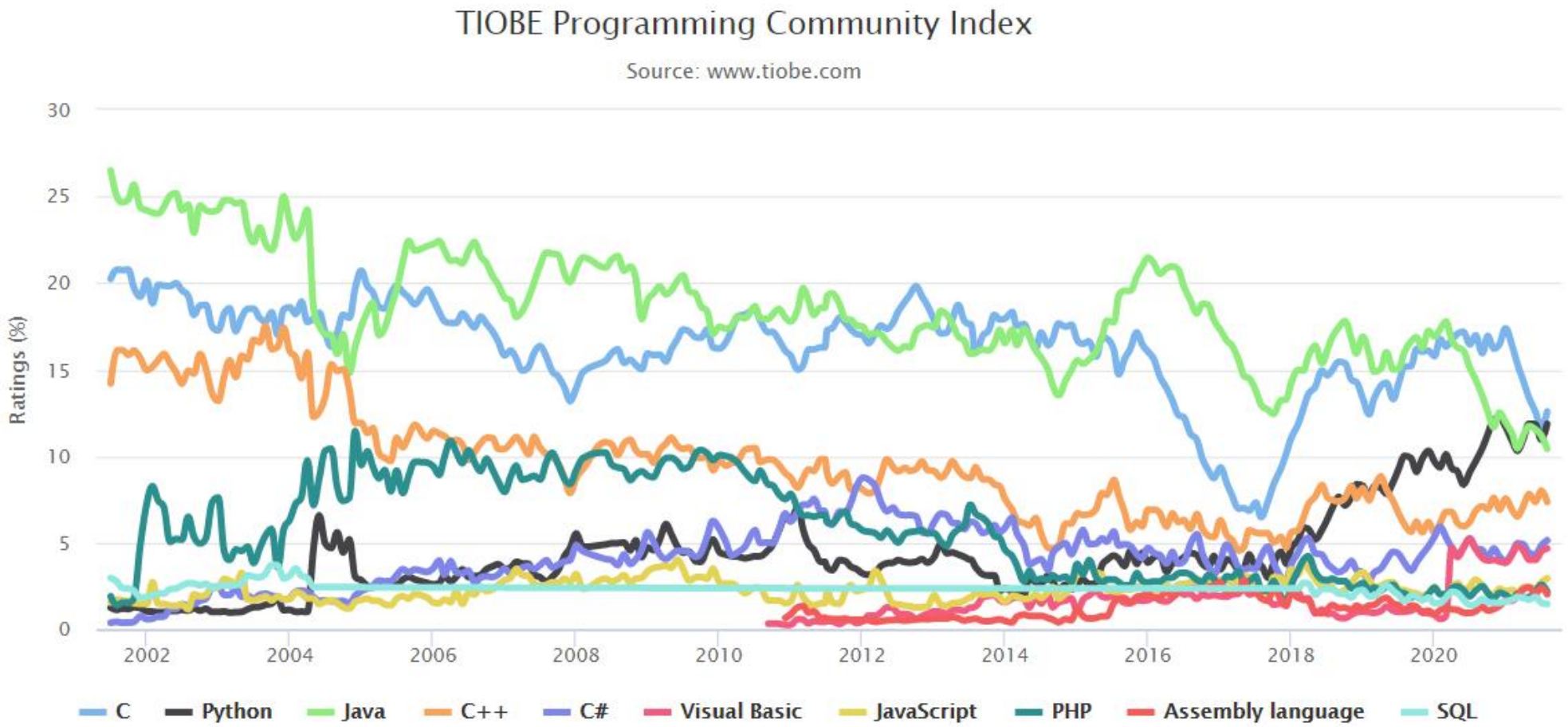
Python语言的发展历史

- Python 2.0版本于2000年10月发布。
- 在2008年12月，Python 3.0发布，此版本没有完全兼容之前的Python 2.0
- Python也因此分为了Python 3.5派系和Python 2.7派系两大阵营



Python语言的TIOBE INDEX

- TIOBE INDEX:编程语言流行程度排行榜



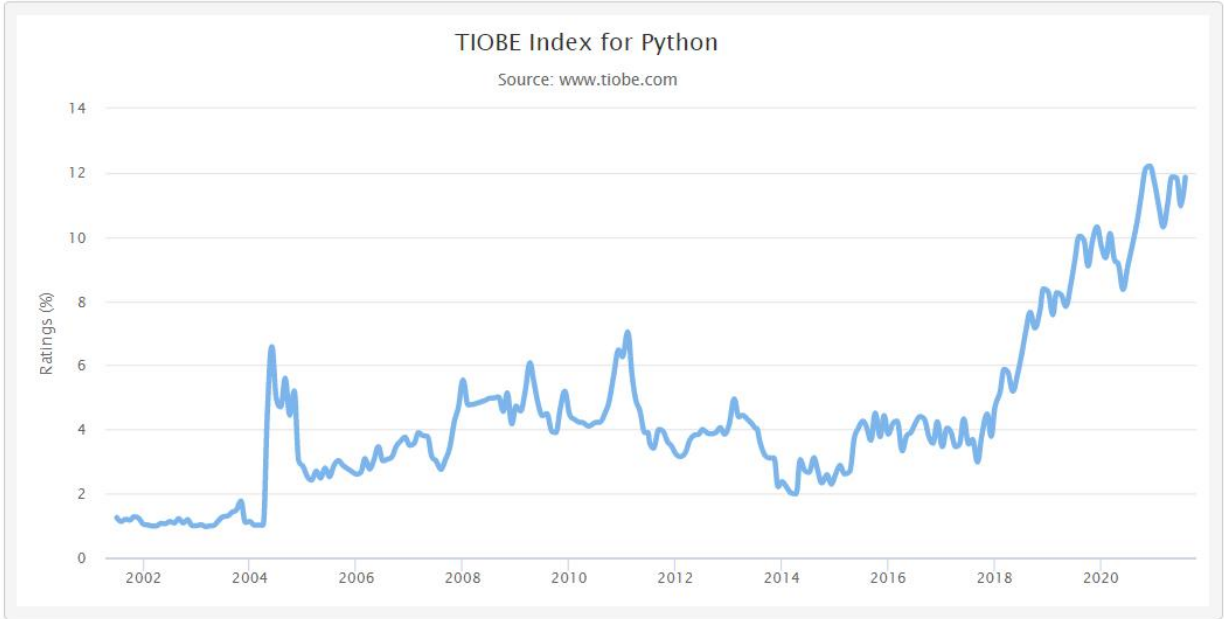
Python语言的TIOBE INDEX

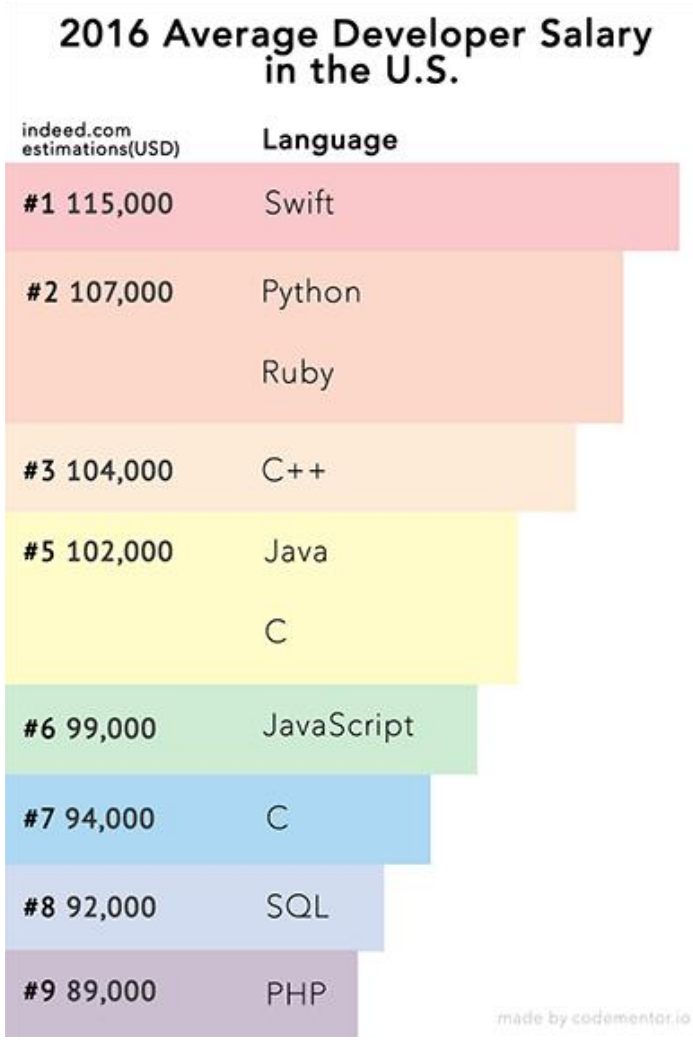
- Python曾多次被TIOBE排行榜评为“年度编程语言”
- 现已成为了第三大流行编程语言

Highest Position (since 2001): #2 in Aug 2021

Lowest Position (since 2001): #13 in Feb 2003

Language of the Year: 2007, 2010, 2018, 2020





Python语言的特点

- 优点一：优雅、简单、明确

  （减少花哨、晦涩或以“炫技”为目的的代码）

- 让数据分析师们摆脱了程序本身语法规则的泥潭，更快的进行数据分析

C语言

```c
#include<stdio.h>
main()
{
    printf("Hello World");
}
```

Python语言

```
print "Hello World"
```

```
Hello World
```

- 优点二：强大的标准库
- 完善的基础代码库，覆盖了网络通信、文件处理、数据库接口、图形系统、XML处理等大量内容，被形象地称为“内置电池”（batteries included）
- Python使用者——“调包侠”

```
import this
```

```
The Zen of Python, by Tim Peters

Beautiful is better than ugly.
Explicit is better than implicit.
Simple is better than complex.
Complex is better than complicated.
Flat is better than nested.
Sparse is better than dense.
Readability counts.
Special cases aren't special enough to break the rules.
Although practicality beats purity.
Errors should never pass silently.
Unless explicitly silenced.
In the face of ambiguity, refuse the temptation to guess.
There should be one-- and preferably only one --obvious way to do it.
Although that way may not be obvious at first unless you're Dutch.
Now is better than never.
Although never is often better than *right* now.
If the implementation is hard to explain, it's a bad idea.
If the implementation is easy to explain, it may be a good idea.
Namespaces are one honking great idea -- let's do more of those!
```

- 优点三：良好的可扩展性
- 大量的第三方模块，覆盖了科学计算、Web开发、数据接口、图形系统等众多领域，开发的代码通过很好的封装，也可以作为第三方模块给别人使用。如Pandas、Numpy、Seaborn、Scikit-learn等等
- 优点四：免费、开源

Python语言的缺点

- 缺点一：运行速度慢
- 缺点二：加密难
- 缺点三：缩进规则
- 缺点四：多线程灾难

```
if True:
print "right"
```

```
  File "<ipython-input-1-ab7d6f176ce6>", line 2
    print "right"
        ^
IndentationError: expected an indented block
```

```
if True:
    print "right"
```

```
right
```

```
junyilu ··· misc python gil
junyilu ··· misc python gil
junyilu ··· misc python gil python2.7 ./single_thread.py
Total time: 11.4724829197
junyilu ··· misc python gil python2.7 ./multi_thread.py
Total time: 16.1935360432
junyilu ··· misc python gil
```

Python语言与Java





- 动态类型和静态类型
- Python中一切皆对象
- 括号与缩进
- 应用领域

Python语言与R语言



- 机器学习的一把利器
- 可读性强，便于上手
- 灵活性强：可与其他如Web应用程序进行整合



- 以统计推断为导向
- 数据分析之外的领域有所限制
- 包凌乱且一致性较差

Python语言与R语言的应用场景对比





- 网络爬虫
- 连接数据库
- 内容管理系统
- API构建

- 统计分析
- 互动式图标/面板

运行环境及安装

- 推荐使用Anaconda进行Python安装、环境配置及工具包管理
- Ipython
- qtconsole
- PyCharm
- Jupyter Notebook
- Spyder

Python集成开发环境

- IDLE：Python安装包内置开发环境
- Thonny，Python IDE for beginners
- 网址是 https://thonny.org/
- jupyter:安装Anaconda自带的高级IDE，数据科学家首选开发环境
- PyCharm：由著名的JetBrains公司开发，带有一整套可以帮助用户在使用Python语言开发时提高其效率的工具。程序员的开发环境

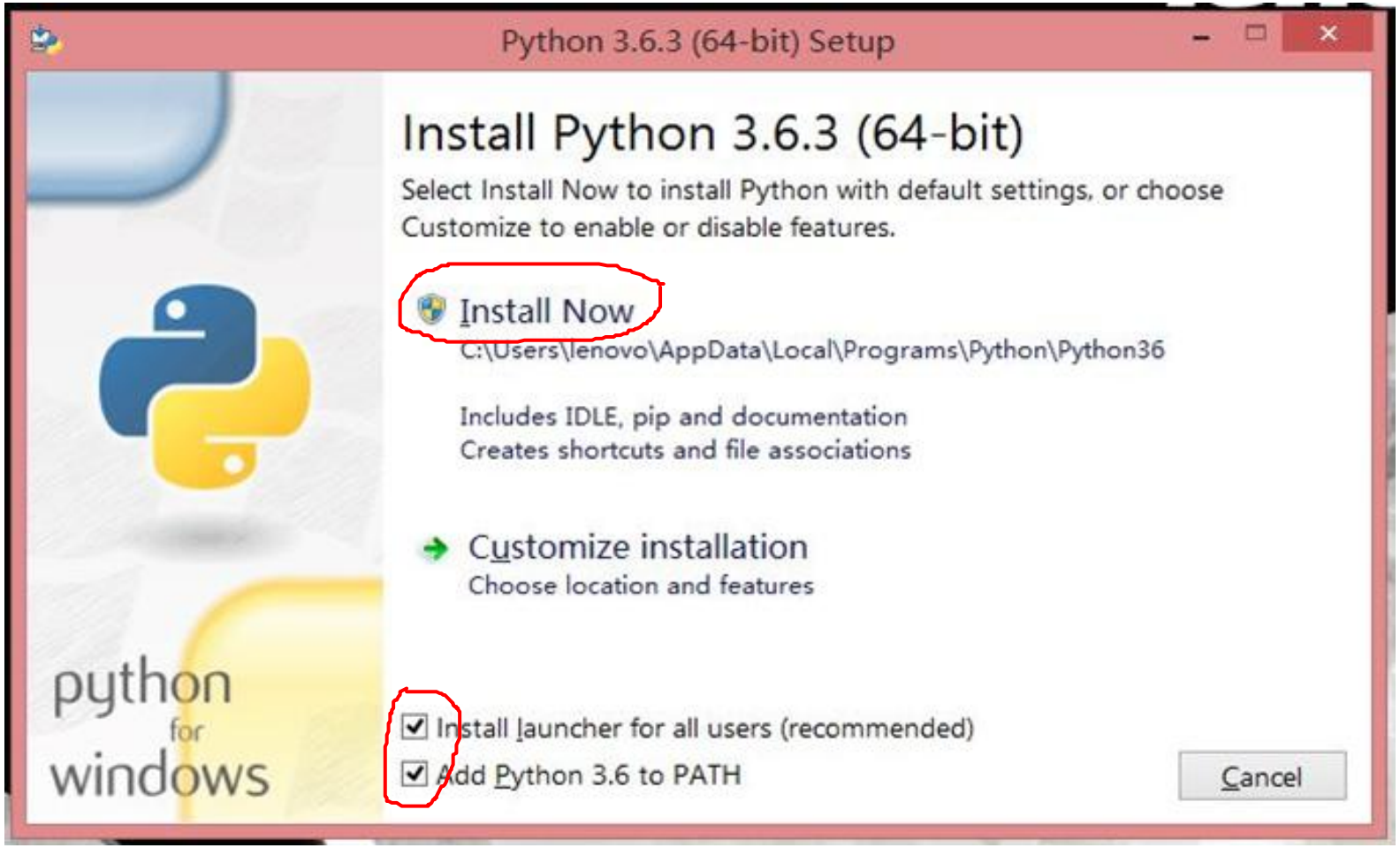
Python IDLE开发环境

- Python： https://www.python.org/downloads
- 选择Python版本
- 选择操作系统

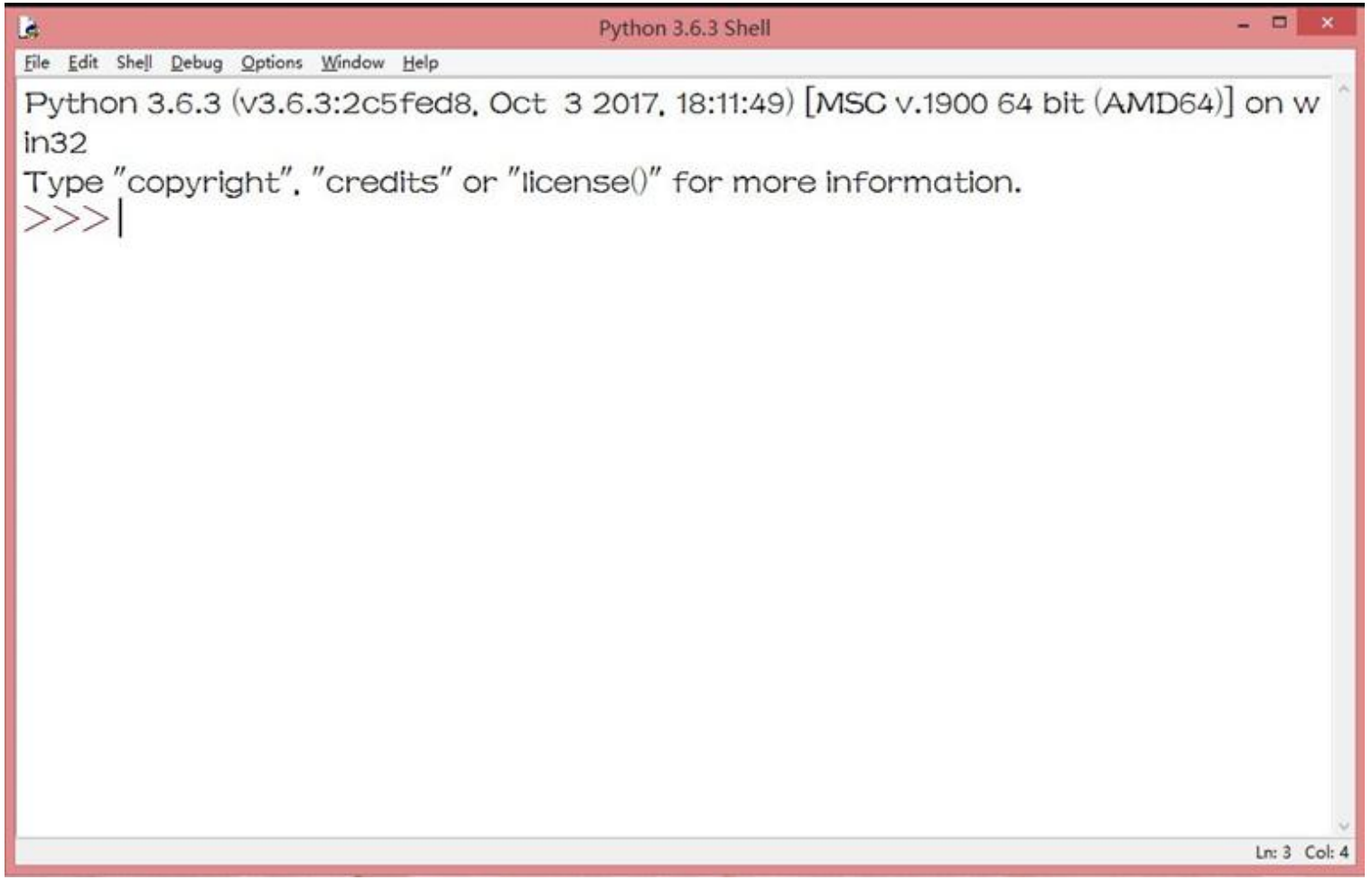
Files

Version	Operating System	Description	MD5 Sum	File Size	GPG
Gzipped source tarball	Source release		9de6494314ea199e3633211696735f65	22710891	SIG
XZ compressed source tarball	Source release		1325134dd525b4a2c3272a1a0214dd54	16992824	SIG
Mac OS X 64-bit/32-bit installer	Mac OS X	for Mac OS X 10.6 and later	9fba50521dffa9238ce85ad640abaa92	27778156	SIG
Windows help file	Windows		17cc49512c3a2b876f2ed8022e0afe92	8041937	SIG
Windows x86-64 embeddable zip file	Windows	for AMD64/EM64T/x64, not Itanium processors	d2fb546fd4b189146dbefeba85e7266b	7162335	SIG
Windows x86-64 executable installer	Windows	for AMD64/EM64T/x64, not Itanium processors	bee5746dc6ece6ab49573a9f54b5d0a1	31684744	SIG
Windows x86-64 web-based installer	Windows	for AMD64/EM64T/x64, not Itanium processors	21525b3d132ce15cae6ba96d74961b5a	1320128	SIG
Windows x86 embeddable zip file	Windows		15802be75a6246070d85b87b3f43f83f	6400788	SIG
Windows x86 executable installer	Windows		67e1a9bb336a5eca0efcd481c9f262a4	30653888	SIG
Windows x86 web-based installer	Windows		6c8ff748c554559a385c986453df28ef	1294088	SIG

安装选项

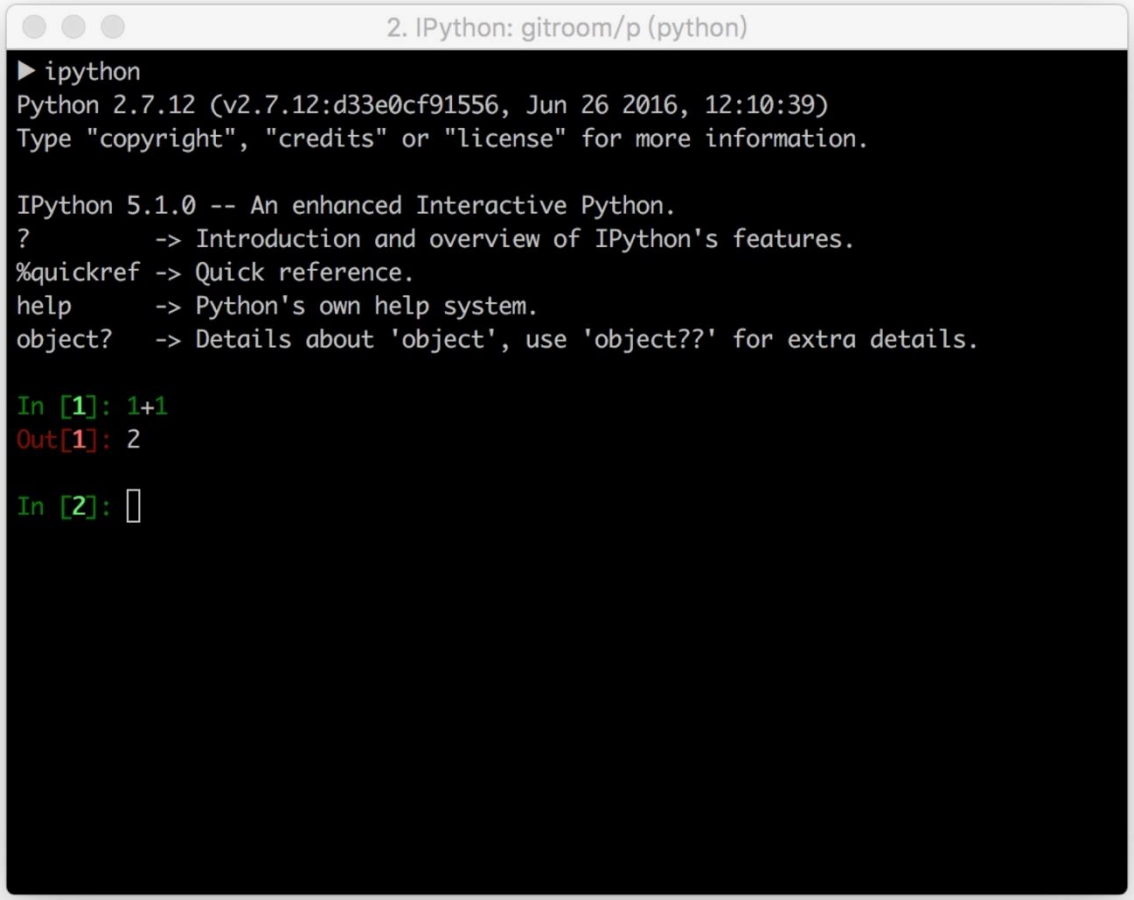

Python 3.6.3 (64-bit) Setup
Install Python 3.6.3 (64-bit)
Select Install Now to install Python with default settings, or choose
Customize to enable or disable features.
Install Now
C:\Users\lenovo\AppData\Local\Programs\Python\Python36
Includes IDLE, pip and documentation
Creates shortcuts and file associations
Customize installation
Choose location and features
python
for
windows
Install launcher for all users (recommended)
Add Python 3.6 to PATH
Cancel

运行Python IDLE开发环境

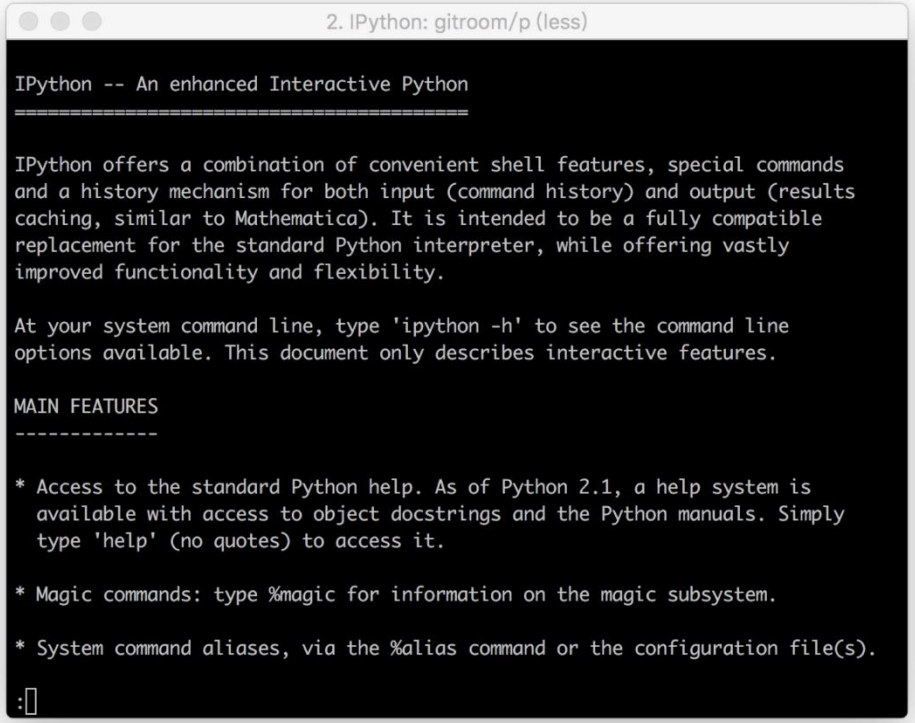


交互式计算和开发环境：IPython

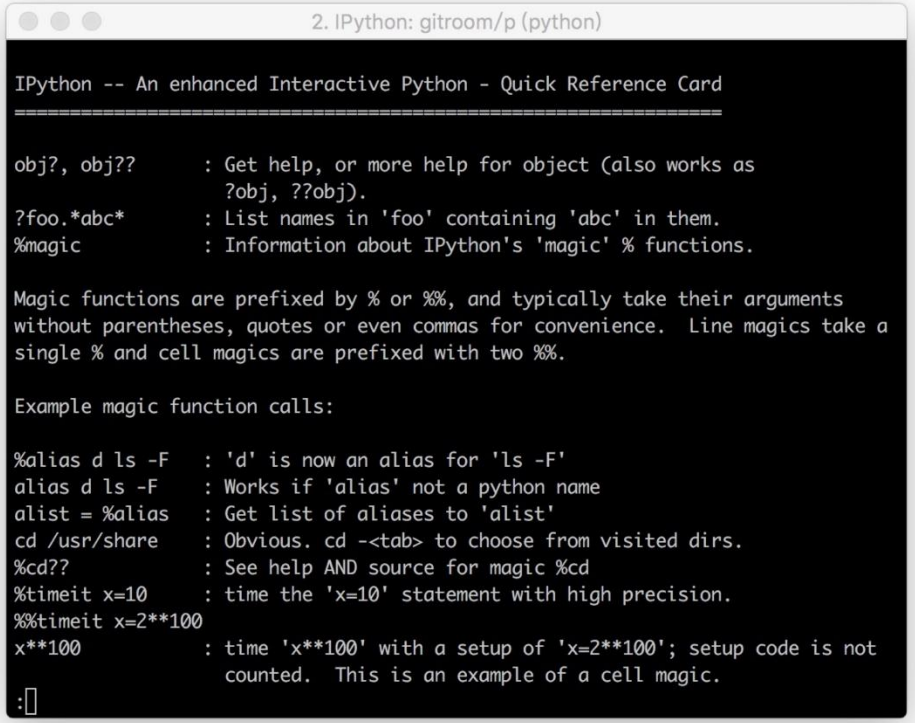
- IPython鼓励一种“执行-探索”（execute-explore）的工作模式
- 输入代码之后，按下回车，便会立即得到代码运行结果



交互式计算和开发环境：IPython

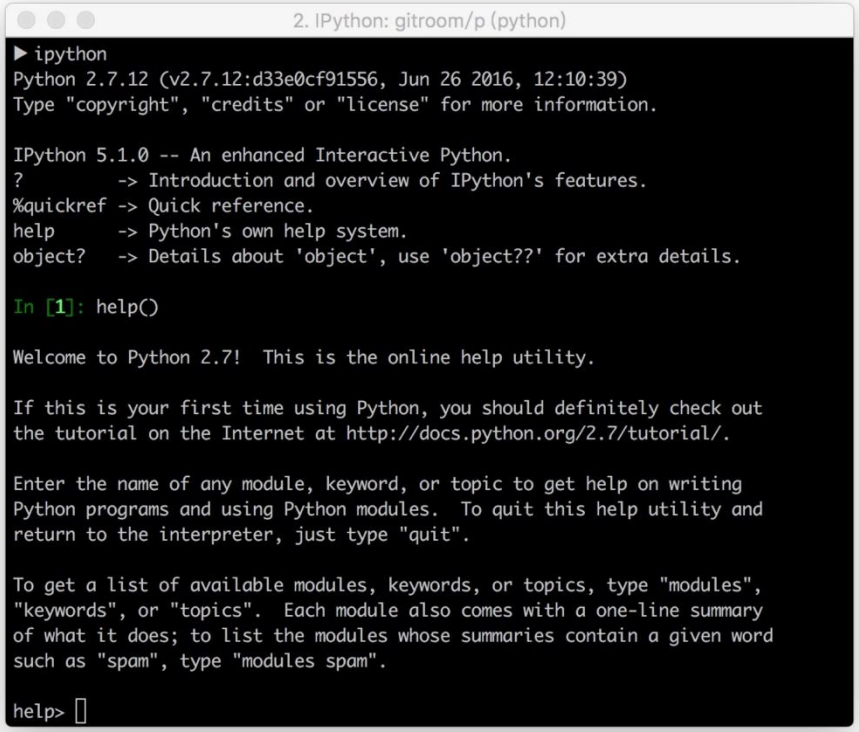


- 输入“?”获得IPython的详细介绍

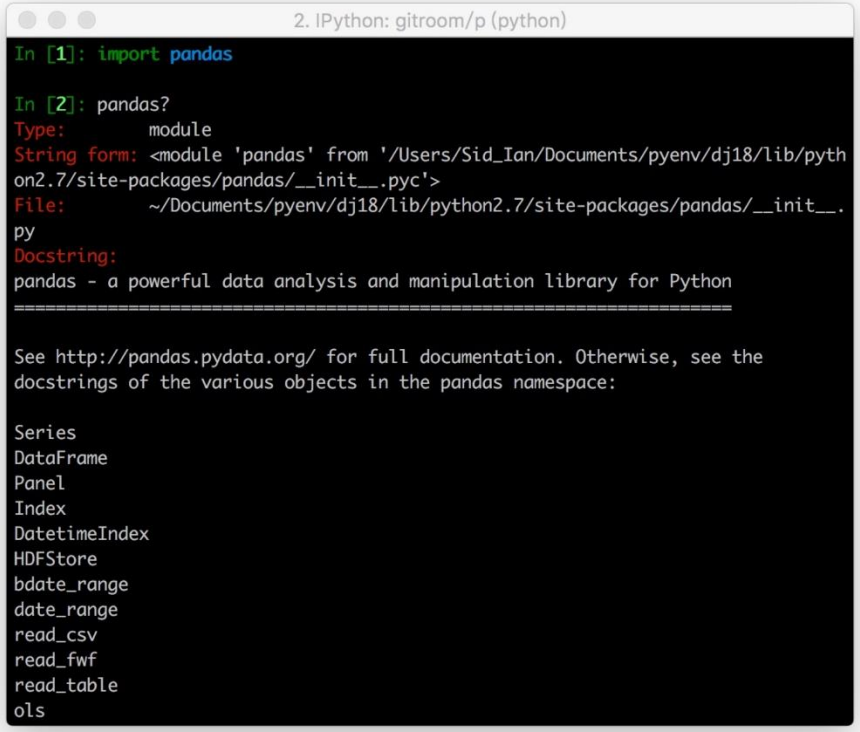


- 输入“%quickref”获得IPython的快速参考

交互式计算和开发环境：IPython



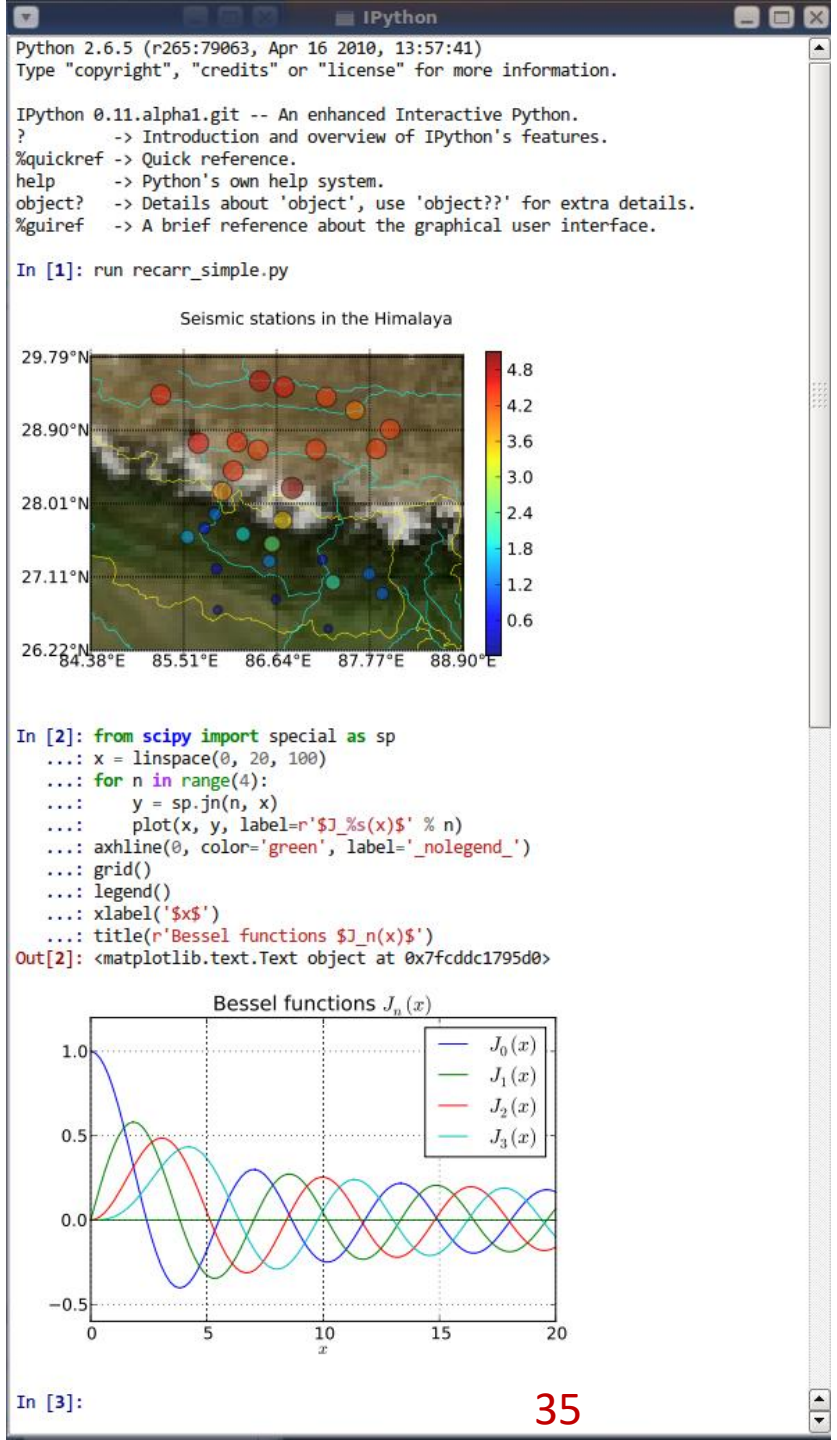
- 输入“help()”查看IPython的帮助文档



- 使用内省“pandas?”查看Pandas的帮助文档

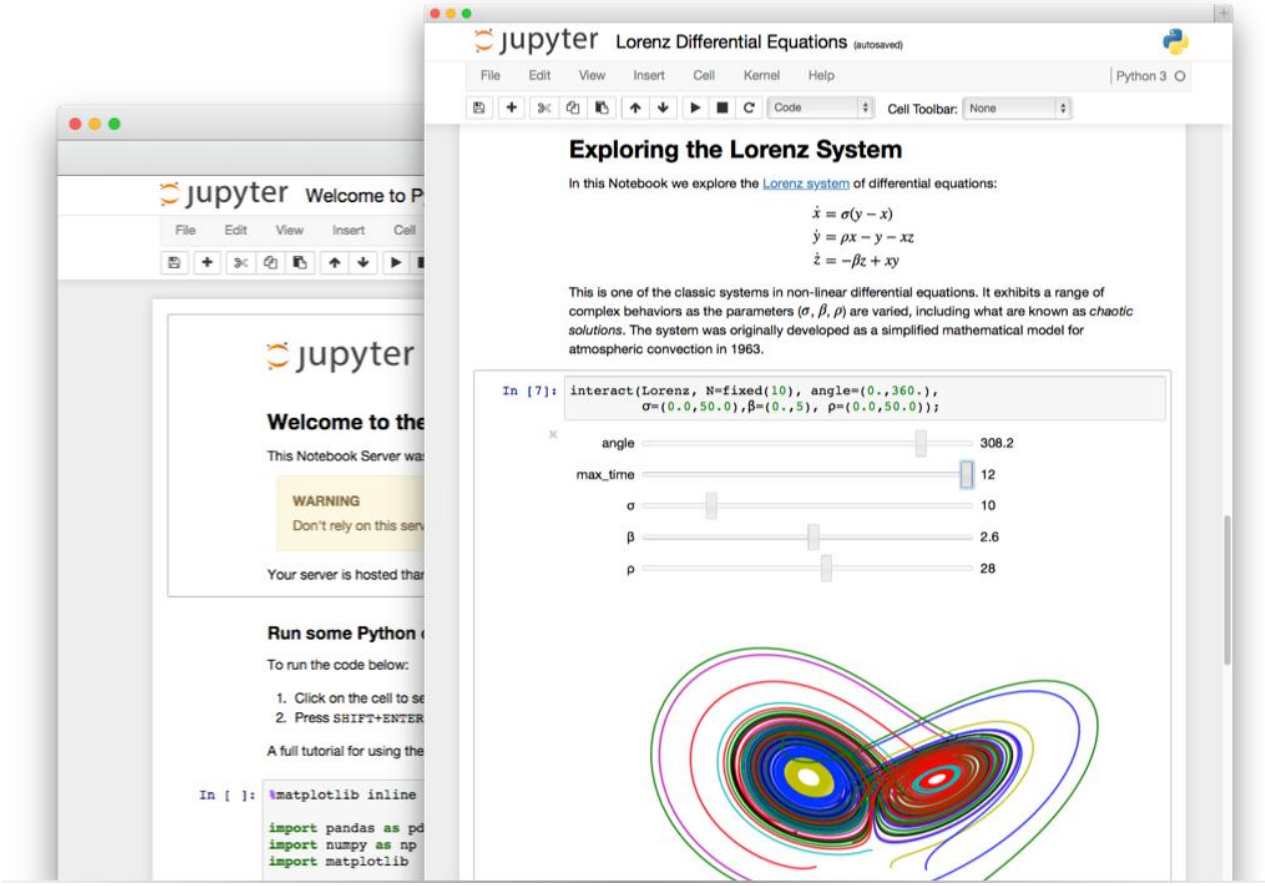
基于Qt框架的GUI控制台——qtconsole

- 为终端应用程序提供诸如内嵌图片、多行编辑、语法高亮之类的富文本编辑功能
- 启动命令：jupyter qtconsole
- 缺点：功能少、用户友好性不够



Jupyter Notebook

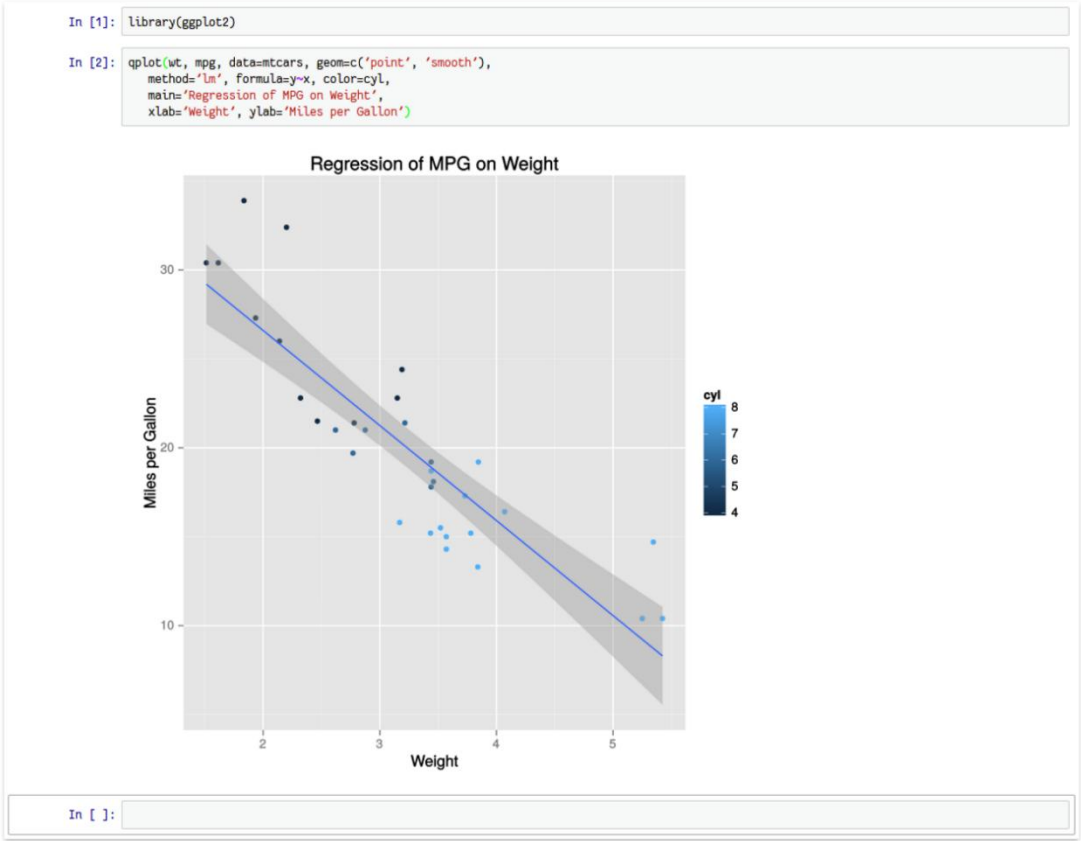
- Julia+Python+R = Jupyter
- 基于Web技术的交互式计算文档格式
- 支持Markdown和Latex语法
- 支持代码运行、文本输入、数学公式编辑、内嵌式画图和其他如图片文件的插入，是一个对代码友好的笔记本



Jupyter Notebook

- Jupyter支持包含R kernel（支持R语言）
- 启动命令：jupyter notebook

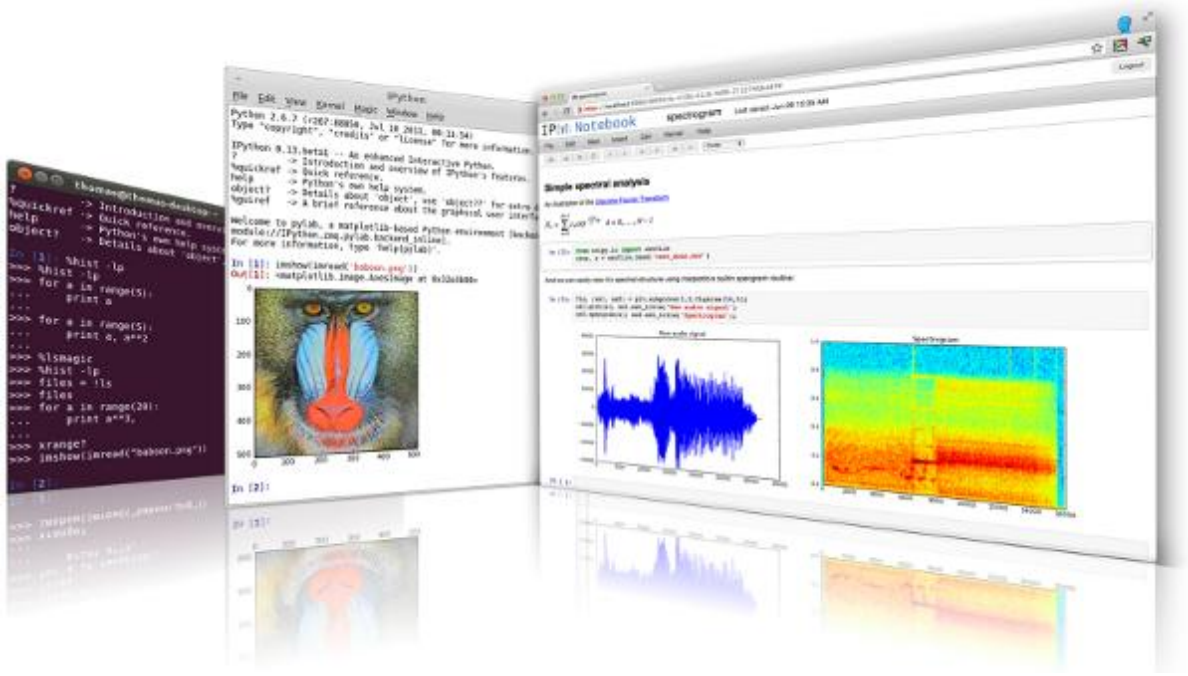
R kernel for Jupyter



Jupyter Notebook

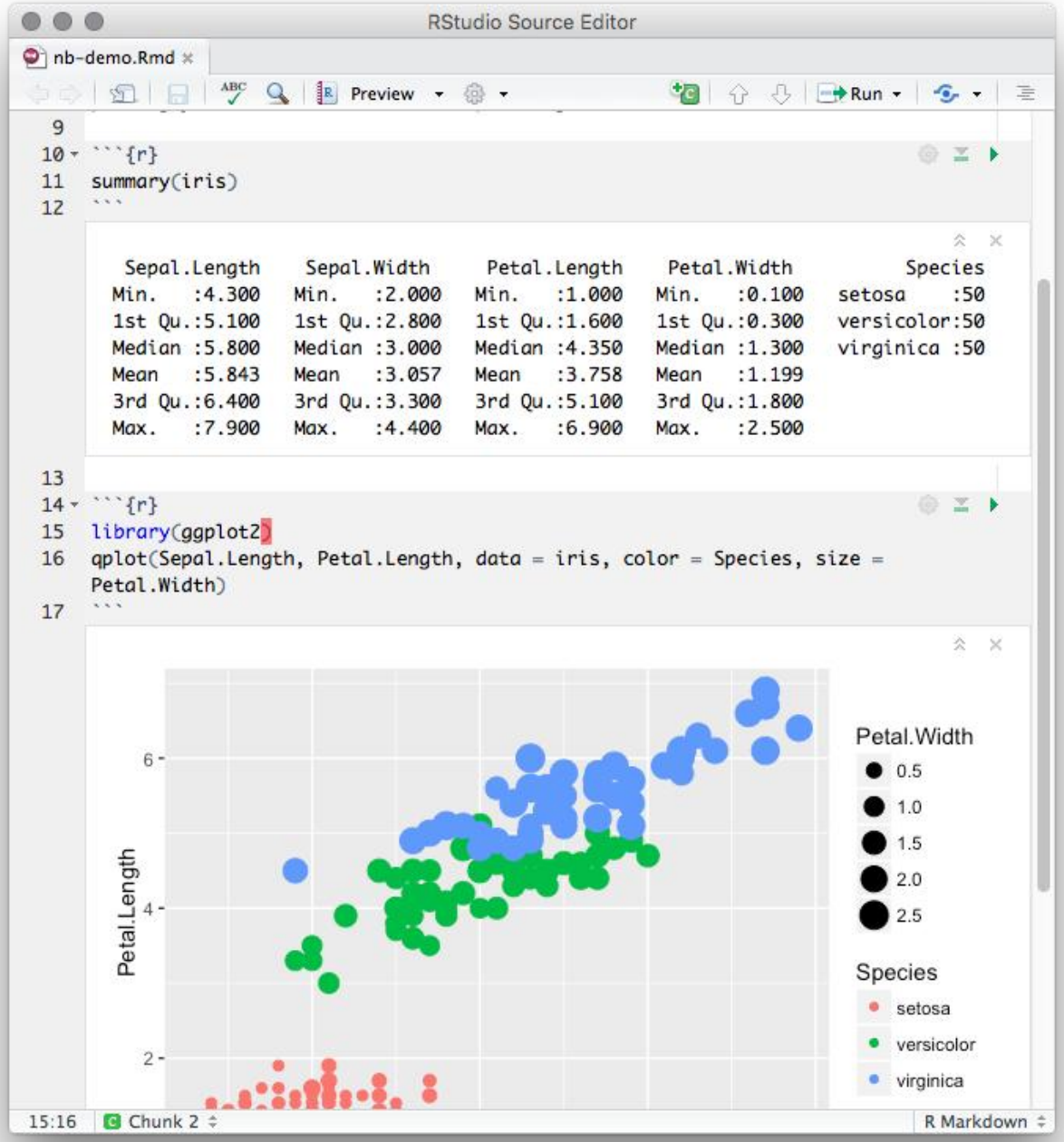
- 推荐使用Jupyter Notebook进行数据分析，并将自己数据分析的思考过程写在其中，方便之后整理思路以及向别人展示数据分析结果

从左至右分别是默认Python终端、IPython和Jupyter Notebook



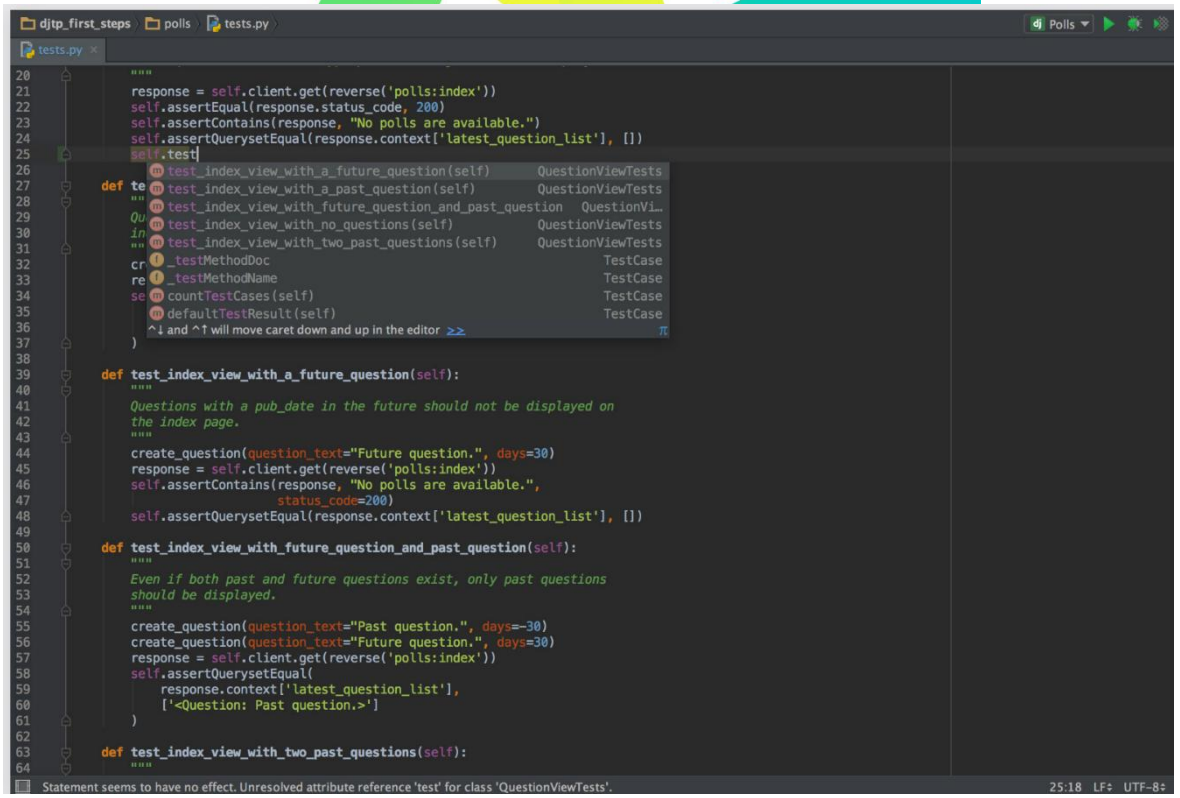
R Notebook

- 2016年10月，R语言的集成环境RStudio也拥有了自己的R Notebook了

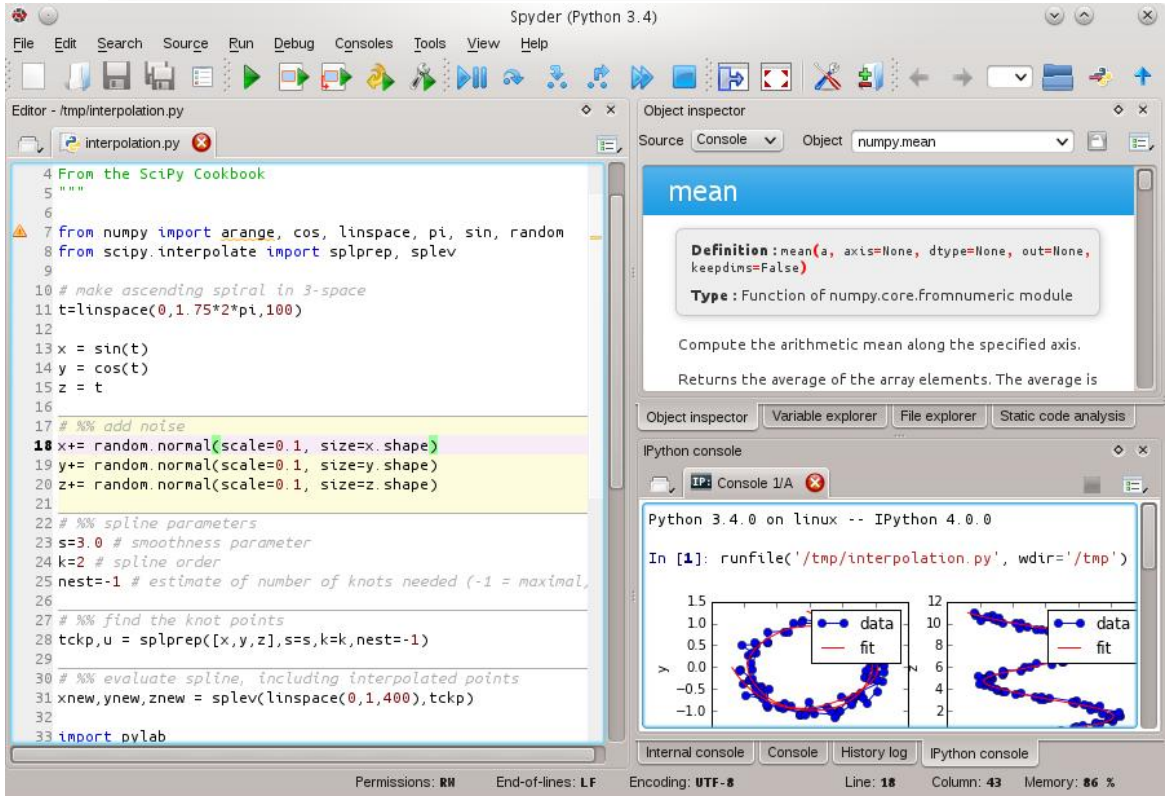


集成开发环境IDE——PyCharm & Spyder

PyCharm
Python IDE
for Professional Developers

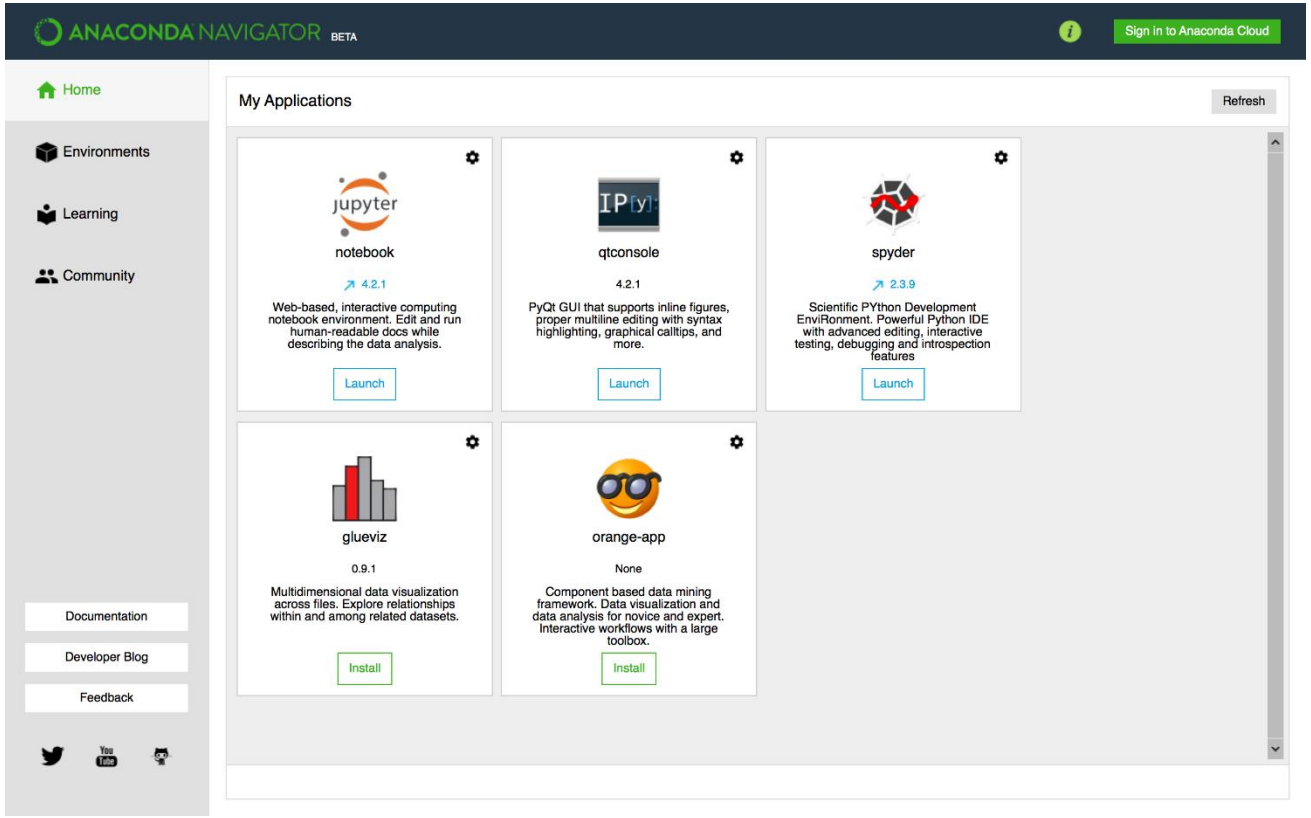


Spyder IDE
The Scientific PYthon Development EnviRonment



数据科学计算平台——Anaconda

- “leading open data science platform powered by Python
- 自动配置Python环境，下载并安装Jupyter Notebook、qtconsole和集成开发环境Spyder
- 包管理器 conda



总结

- Python语言的诞生与发展历史
- Python语言的特点：优雅、简单、易学
- Python语言的运行环境：IPython、Jupyter Notebook、 Spyder…
- Python语言的数据科学计算平台：Anaconda

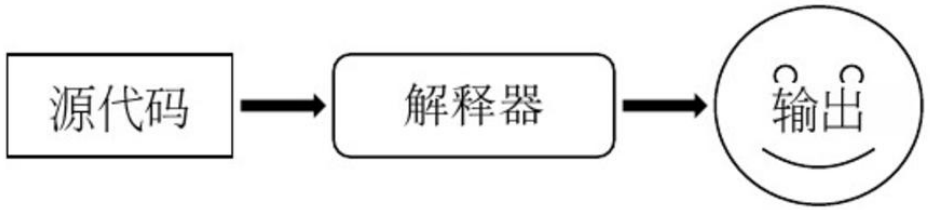


第二部分 基本概念

变量、注释、print函数、数据类型
算术运算符、类型转换

解释型语言Python

- Python语言是一种**解释型**、面向对象、动态数据类型的高级程序设计语言



解释型语言的执行方式



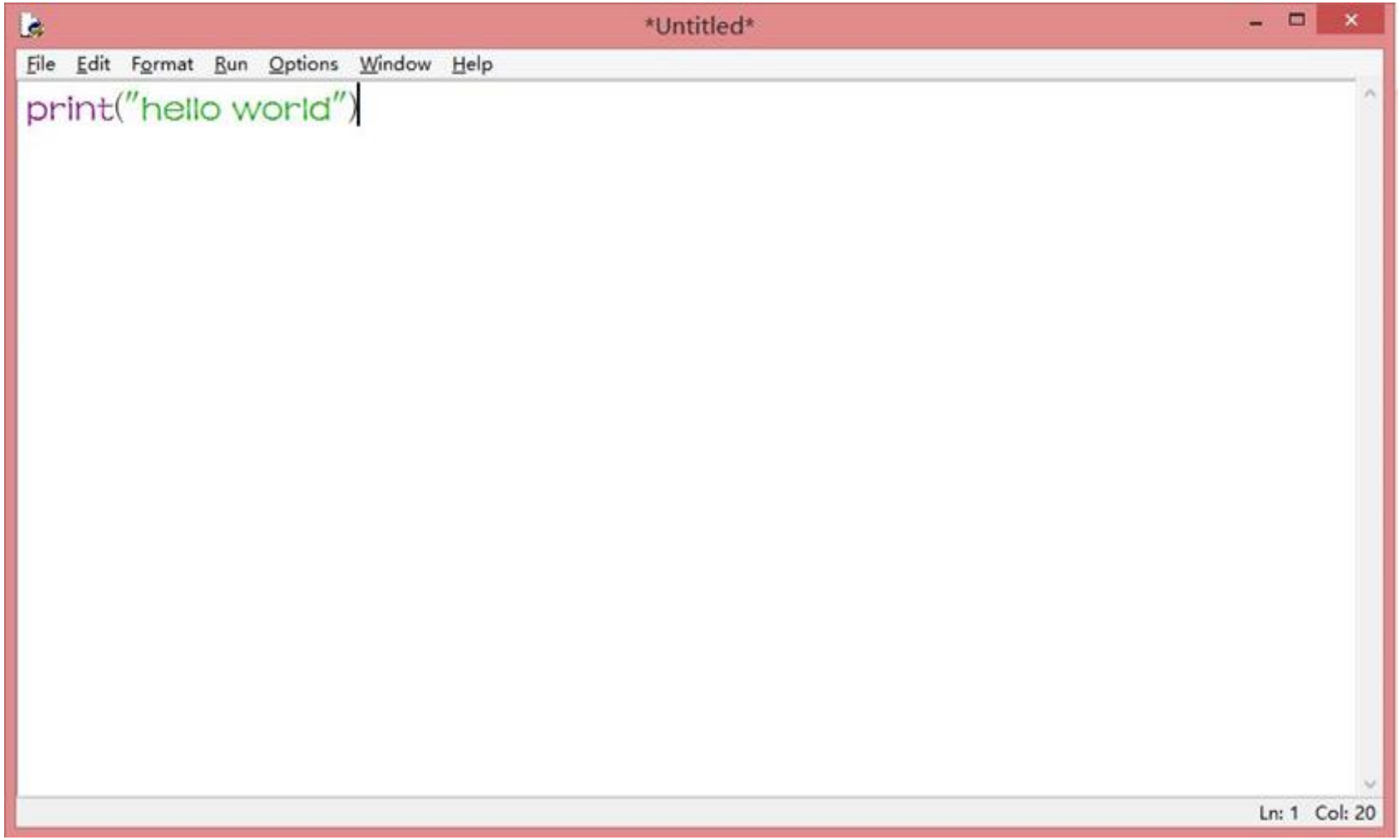
编译型语言的执行方式

编译型	解释型	混合型
c	JavaScript	JAVA
c++	Python	C#
GO	Ruby	
Swift	PHP	
Object-C	Perl	
Pascal	Erlang	

运行程序:交互式解释器执行

- >>>3+5*6
- 33
- >>>print(“hello world”)
- hello world

运行程序: File|New,Save,Run



运行程序:命令行环境运行

- d:>python hello.py
- hello.py是python程序，放在d盘根目录
- hello.py:

```
print(“hello world”)
```

Python代码规范

（1）缩进

Python程序是依靠代码块的缩进来体现代码之间的逻辑关系的

（2）注释

方法一：以#开始，表示本行#之后的内容为注释。

```
#循环输出0到9数字
for i in range(10):
    print (i, end=' ')
```

方法二：包含在一对三引号'''……'''或"""……"""之间且不属于任何语句的内容将被解释器认为是注释。

Python代码规范

（3）每个import只导入一个模块。而不要一次导入多个模块。

（4）如果一行语句太长，可以在行尾加上反斜杠“\”来换行分成多行，但是更建议使用括号来包含多行内容。

（5）必要的空格与空行

运算符两侧、函数参数之间、逗号两侧建议使用空格分开。不同功能的代码块之间、不同的函数定义之间建议增加一个空行以增加可读性。

（6）常量名所有字母大写，由下划线连接各个单词。类名首字母大写。

使用帮助

- 使用格式如下：help(对象)
- 下面分3种情况进行说明。
- 1．查看内置函数和类型的帮助信息
- >>> help(max)
- 2．查看模块中的成员函数信息
- >>> import os
- >>> help(os.fdopen)
- 上例查看os模块中的fdopen成员函数信息
- 3．查看整个模块的信息
- 使用help(模块名)就能查看整个模块的帮助信息。注意先import导入该模块。例如：查看math模块方法：
- >>> import math
- >>> help(math)

Python关键字

- python一些特殊的组合，是所谓的关键字。关键字不允许作为标识符。
- Python关键字：

False	def	if	raise
None	del	import	return
True	elif	in	try
and	else	is	while
as	except	lambda	with
assert	finally	nonlocal	yield
break	for	not	
class	from	or	
continue	global	pass	

id函数

- Python变量有一个非常重要的性质：变量是将名字和对象关联起来。赋值操作并不会实际复制值，它只是为数据对象取个相关的名字。名字是对象的引用而不是对象本身
- id是Python的内置函数，显示对象的地址

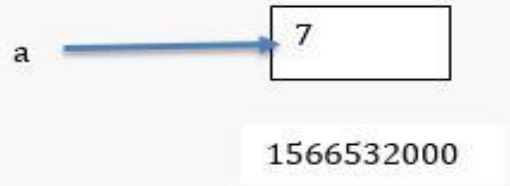
id函数用法

```
>>> id(a)

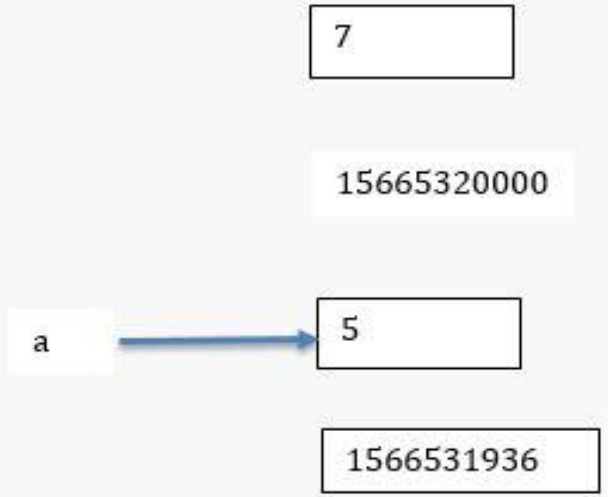
1566532000

>>> id(7)

1566532000
```




```
>>> a=5
>>>a
5
>>>id(a)
1566531936
>>>id(5)
1566531936
>>> id(7)
1566532000
```



标识符和变量

- 标识符是指用来标识某个实体的一个符号，在不同的应用环境下有不同的含义
- 标识符由字母、下划线和数字组成，且不能以数字开头
- python中的标识符是区分大小写的，Andy与andy是不同的标识符
- 如：my_test
  _123

常量和变量

- 常量就是不能改变的量，比如常用的数学常数3.14159就是一个常量
- 变量就是程序为了方便地引用内存中的值而为它取的名称。Python变量名是大小写敏感的
- >>>a=7 “=“是赋值号
- >>>a
- 7
- 7是一个对象，可以通过变量a引用这个对象

变量赋值

- 通过赋值运算符 = 变量名和想要赋予变量的值连接起来，变量的赋值操作就完成了声明和定义的的过程，在其他语言中需要制定类型；
- 同一变量可以反复赋值，而且可以是不同类型的变量，这也是Python语言称之为动态语言的原因

```
shot_id=1
print(shot_id,type(shot_id))
shot_id='1'
print(shot_id,type(shot_id))
```

```
1 <class 'int'>
1 <class 'str'>
```

常量

- 常量表示“不能变”的变量
- Python中是没有常量的关键字的，只是我们常常约定使用大写字母组合的变量名表示常量，也有不要对其进行赋值”的提醒作用

```
PI = 3.14159265
```

注释

- 如同我们在看书时做笔记一样
- Python语言会通过注释符号识别出注释的部分，将它们当做纯文本，并在执行代码时跳过这些纯文本
- 在Python语言中，使用 # 进行行注释

```
# 这是在代码之前的一个注释!
shot_id = 1 # 这是在代码所在行的一个注释!
# 这是在代码之后的一个注释!
```

- 多行注释使用连续单个单引号或者双引号

```
'''
这是第一行注释!
这是第二行注释!
'''


"""
这是第一行注释!
这是第二行注释!
"""
```

输入及输出函数

- 输入函数：input()
- input从键盘输入一个字符串。‘9’表示是一个字符串,它的ASCII码值是57
- >>>a=input()

  9

  >>>a

  ‘9’

print函数

- 在Python 2.x版本中，同时兼容 print 和 print()
- 在Python 3.x版本中，print 函数为带括号的 print()
- 如果想要看变量的值，则直接在 print 后面加上变量名即可。如果是想要输出提示信息，如一句话，那我们需要将提示信息用‘单引号包裹起来（这使得内容构成一个字符串）

```
>>> shot_id=1
>>> print(shot_id)
1
>>> print('投篮ID!')
投篮ID!
>>>
```

- 使用逗号,隔开变量与其他剩余内容，则 *print* 在输出时会依次打印各个字符串或变量，遇到逗号,时会输出一个空格

```
print('投篮的ID是',shot_id)
```

```
投篮的ID是 2
```

- *print* 函数不仅可以打印变量值，也可以打印计算结果

```
shot_id = 2
print (shot_id)  #此行打印变量’shot_id’的值
print (shot_id + 3)  #此行打印变量’shot_id’加上3的结果
print ('shot_id + 3 =' ,shot_id + 3)  #此行将等式打印出来
```

```
2
5
shot_id + 3 = 5
```

数据类型

- Python语言的数据类型包括整型、浮点型、字符串、布尔型和空值
- 整型（int）
- 整型的取值为整数，有正有负，如 2，-666，666 等。在投篮数据集中，shot_id、game_event_id、game_id、loc_x、loc_y、minutes_remaining、period、playoffs、seconds_remaining、shot_distance、shot_made_flag、team_id都是整型变量

浮点型（float）

- 浮点型的取值为小数，当计算有精度要求时被使用,由于小数点可以在相应的二进制的不同位置浮动，故而称为浮点数
- 如 3.14，-6.66 等，但是如果是非常大或者非常小的浮点数，就需要使用科学计数法表示，用 e 代替 10 。投篮数据集中的 lat 和lon 为浮点型变量。

```
3.14e9
```

```
3140000000.0
```

字符串（str）

- 字符串（str）
- 字符串是以两个单引号或两个双引号包裹起来的文本
- 字符串 Jump Shot 包括 J，u，m，p，空格，S，h，o，t 这9个字符

```
>>> print("team name")
team name
```

- 转义字符：字符串里常常存在一些如换行、制表符等有特殊含义的字符，这些字符称之为转义字符
- 比如 \n 表示换行，\t 表示制表符，Python还允许用 r“ ” 表示“ ”内部的字符串默认不转义

```
>>> print("The team name is Los Angeles Lakers.")
>>> print("The team name is \t Los Angeles Lakers.")
>>> print("The team name is \n Los Angeles Lakers.")
>>> print(r"The team name is \n Los Angeles Lakers.")
The team name is Los Angeles Lakers.
The team name is Los Angeles Lakers.
The team name is
Los Angeles Lakers.
The team name is \n Los Angeles Lakers.
```

布尔型（bool）

- 布尔型（bool）
- 布尔型只有 True 和 False 两种值。比较运算和条件表达式都会产生 True 或 False

```
print ('shot_id equals ' ,shot_id)
print (" 'shot_id is more than 3' is" ,shot_id > 3)
print " 'shot_id is less than 3' is" ,shot_id < 3)
print ("'action_type equals Jump shot' is" ,action_type == 'Jump shot')
print ("'action_type is not equal to Jump Shot' is" ,action_type != 'Jump Shot ')
```

```
shot_id equals 2
'shot_id is more than 3' is False
'shot_id is less than 3' is True
'action_type equals Jump shot' is True
'action_type is not equal to Jump Shot' is False
```

- 布尔型（bool）
- 布尔值可以进行 *and* 、 *or* 和 *not* 运算， *and* 和 *or* 运算分别用 & 和 | 表示

and 运算

	True	False
True	True	False
False	False	False

or 运算

	True	False
True	True	True
False	True	False

- 布尔型（bool）
- not 运算为非运算，即把 True 变成 False ， False 变成 True 。

```
print ('True & False equals to ' ,True&False)
print ('True or False equals to' ,True | False)
print ('not True equals to ' ,not True)
print ('not False equals to' ,not False)
```

```
True & False equals to False
True or False equals to True
not True equals to False
not False equals to True
```

空值（NoneType）

- 空值是Python里一个特殊的值，用 *None* 表示，一般用 *None* 填充表格中的缺失值
- 使用 *type()* 函数来获取某值的类型

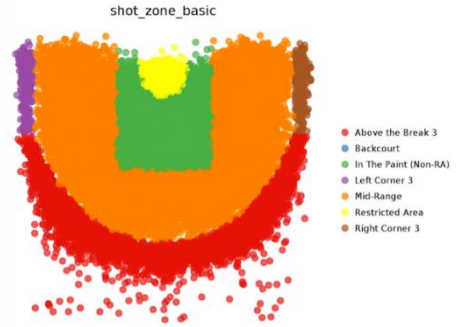
```
print (type( 2 ))
print (type ( 34.0443))
print (type ( ' Jump shot ' ))
print (type ( 'True))
print (type ( None))
```

```
<type 'int'>
<type 'float'>
<type 'str'>
<type 'bool'>
<type 'NoneType'>
```

数据集变量的数据类型总结

变量名	说明	类型	示例	变量名	说明	类型	示例
shot_id	投篮ID	int64	2	seconds_remaining	比赛剩余秒数	int64	32
action_type	细分投篮类型	object	Floating Jump Shot	shot_distance	投篮离篮筐的距离	int64	24
combined_shot_type	综合投篮类型，共六种： Jump Shot 跳投 Layup 上篮 Dunk 扣篮 Tip Shot 补篮 Hook Shot 勾手投篮 Bank Shot 擦板投篮	object	Layup	shot_made_flag	命中标记 0 未命中 1 命中	int64	0
game_event_id	赛事ID	int64	318	shot_type	得分类型 2PT Field Goal 2分球 3PT Field Goal 3分球	object	2PT Field Goal
game_id	比赛ID	int64	21501228	shot_zone_area	投篮位置（区域划分）	object	Center(C)
lat	投篮位置纬度	float64	34.0443	shot_zone_basic	投篮位置(基本)	object	Mid-Range
loc_x	横向投篮位置（短边） x	int64	0	shot_zone_range	投篮位置离框距离	object	15-24 ft
loc_y	纵向投篮位置（长边）y	int64	7	team_id	所在队伍ID	int64	1610612747
lon	投篮位置经度	float64	-118.2698	team_name	所在队伍名字	object	Los Angeles Lakers
minutes_remaining	比赛剩余分钟数	int64	4	game_date	比赛日期	object	2016-04-13
period	比赛第几节	int64	3	matchup	对阵形势	object	LAL @ SAS
playoffs	季后赛标识 0 常规赛 1 季后赛	int64	1	opponent	对手	object	SAS
season	赛季	object	2005-2006				





int64、float64分别表示64位的整型和64位的浮点型

日期数据类型

- 还有一种特殊的数据类型——日期类型
- 这种类型数据在金融、交通等领域十分常见，我们将在日期数据的处理这一部分对这一类型的数据分析进行详细讲解



算术运算符

- 二元数学运算符

运算	说明
a + b	a加b
a – b	a减b
a * b	a乘以b
a / b	a除以b
a // b	a除以b后向下圆整，丢弃小数部分
a ** b	a的b次方

- Python 2.x版本与Python 3.x版本（以下简称Python 2和Python 3）的整除是不同的。

Python 2

```
print ('1 / 2=',1 / 2)
print ('1 // 2 =',1 //2)
print ('1 / 2.0 =',1/2.0)
print ('1 // 2.0 =',1 // 2.0)
```

```
1 / 2 = 0
1 // 2 = 0
1 / 2.0 = 0.5
1 // 2.0 = 0.0
```

Python 3

```
from_future__import division
print ('1 / 2=',1 / 2)
print ('1 // 2 =',1 //2)
print ('1 / 2.0 =',1/2.0)
print ('1 // 2.0 =',1 // 2.0)
```

```
1 / 2 = 0.5
1 // 2 = 0
1 / 2.0 = 0.5
1 // 2.0 = 0.0
```

类型转换

- 函数 *int()*、*float()*、*str()* 和 *bool()* 分别用于将变量转换成整型、浮点型、字符串和布尔型变量

```
print('shot_id is' ,type ( shot_id)))
print(type ( float (shot_id) ) ,float ( shot_id))
print(type ( str( shot_id) ) ,str ( shot_id))
print(type ( bool( shot_id ) ) , bool (shot_id))
```

```
shot_id is <type 'int'>
<type 'float'> 2.0
<type 'str'> 2
<type 'bool'> True
```

- 某些变量无法转换成数值型变量

```
print ('action_type is',type ( action_type))
```

```
action_type is <type 'str'>
```

```
print (type( int ( action_type) ) ,int ( action_type))
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
<ipython-input-24-512f5297a0e0> in <module>()
----> 1 print type(int(action_type)),int(action_type)

ValueError: invalid literal for int() with base 10: 'Jump Shot'
```

```
int('2')
```

```
2
```

- 只有在变量值为 0 时，bool 转换的结果才为 False：

print ('action_type is',type (action_type))
print (type(bool (action_type)) ,bool (action_type))

```
action_type is <type 'str'>
<type 'bool'> True
```

```
bool(0)
```

```
False
```

- 除了使用 *type()* 外，我们还可以使用 *isinstance()* 来获得数据类型

```
isinstance('2',str)
```

```
True
```

```
isinstance(2,int)
```

```
True
```

输入数字

- 用int()函数输入数字
- >>>a=int(input())

  9

  >>>a

  9

一行输入多个值

```
>>>m,n=input("请输入多个值：").split()
请输入多个值：3 5
 >>>m
'3'
>>>n
'5'
```

input(“请输入多个值：”)函数中的参数是输出提示字符串

输出函数：print()

- print是输出函数，参数是输出值
- >>> print(3)　　#输出1个数字
- 3
- >>>print(3,7)　#输出2个数字
- 3 7
- >>> b,c=3,4　　#输出1个数字，两个变量
- print(b, c, 5)
- 3 4 5
- 井号“#”常被用作单行注释符号，在代码中使用“#”时，它右边的任何数据都会被忽略，当做是注释

不换行输出

- 每行输出一个值
- >>>print(3)
- >>>print(4)
- >>>print(5)
- 用end参数，一行输出三个值,
- print(3,end=' ')
- print(4,end=' ')
- print(5,end=' ')

输入三角形的三边长度3,4,5，求这个三角形的面积

```
import math       #引入数学库
a=int(input())
b=int(input())
c=int(input())
s=(a+b+c)/2
#  '*'表示乘，math.sqrt表示开根号
area=math.sqrt(s*(s-a)*(s-b)*(s-c))
print("三角角形的边长：",a,b,c,end=' ')
print("三角角形的面积：",area)
```

画五角形

```
import turtle
turtle.forward(200)
turtle.right(144)
turtle.forward(200)
turtle.right(144)
turtle.forward(200)
turtle.right(144)
turtle.forward(200)
turtle.right(144)
turtle.forward(200)
turtle.done()
```

总结

- 变量、注释、print函数
- 数据类型、算术运算符、类型转换